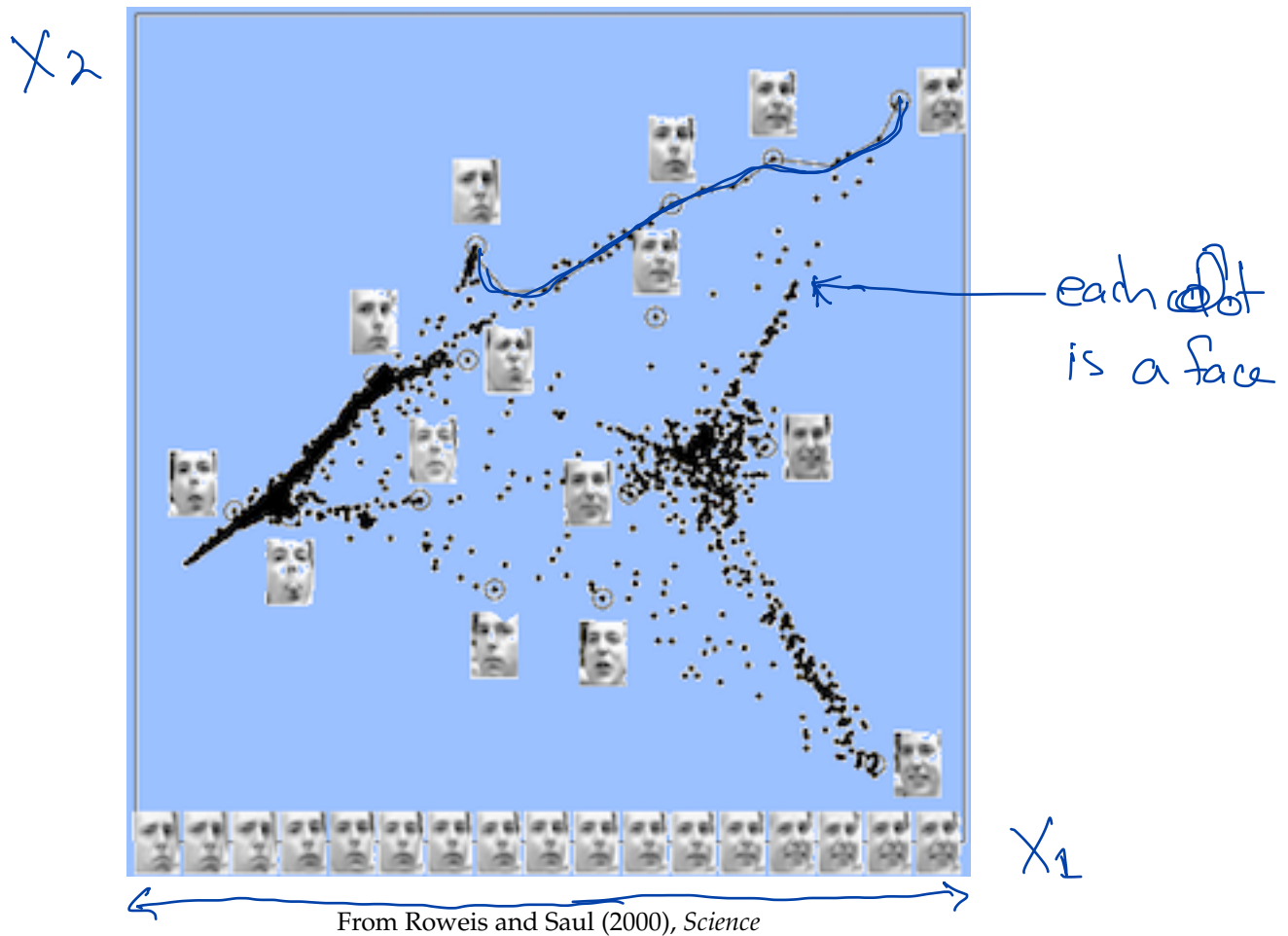




Part 16: Nonlinear Dimension Reduction

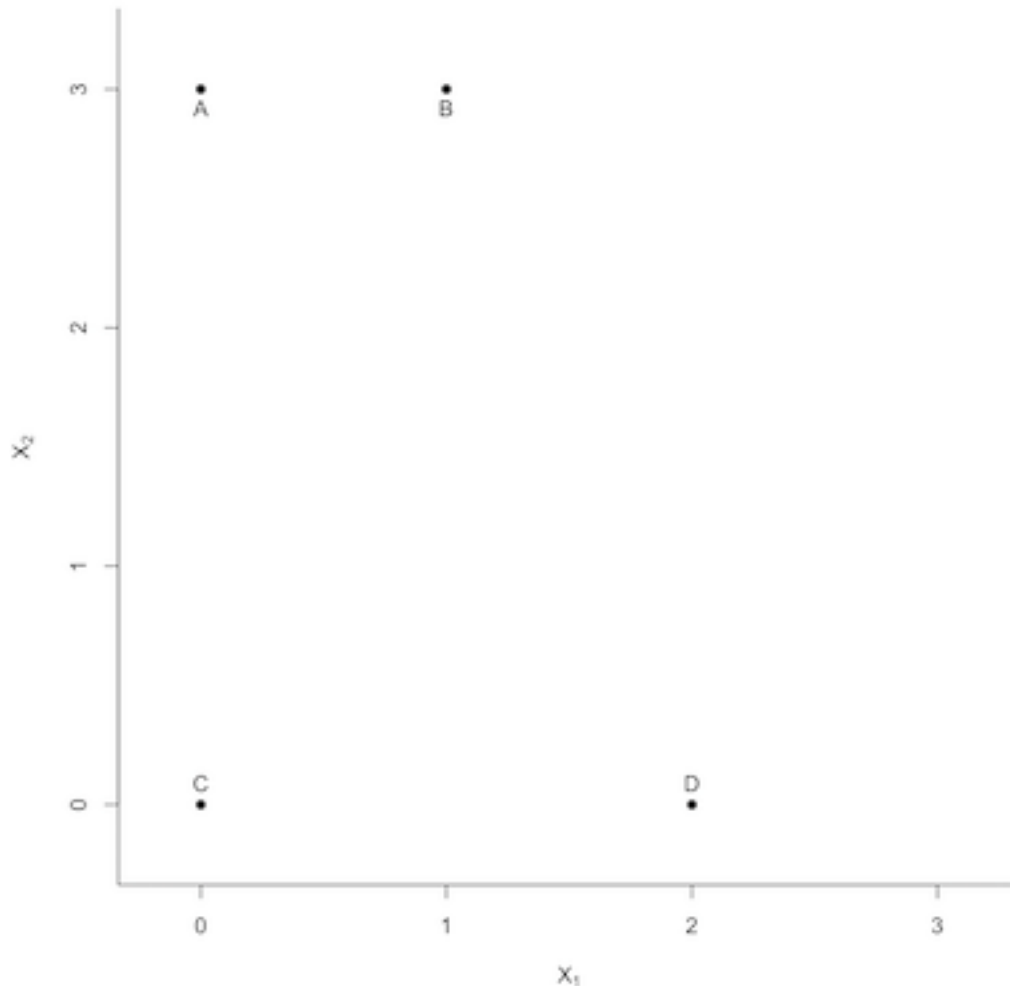
Consider the figure above. In this case, an individual “observation” is an image of a face. These faces are mapped into a two-dimensional space. The faces along the bottom show how the faces evolve along the line in the upper right of the figure.

This illustrates the use of **nonlinear dimension reduction** on high-dimensional data. PCA would not have been a good choice in this case because individual images cannot be approximated by a linear combination of “basis” faces.

Exercise: What is the utility of the representation shown in the previous figure?

We can use the lower dimensional rep. as inputs into formal statistical or ML approach. Avoid the curse of dimensionality.

Consider a synthetic case in which we have four points in two-dimensions, labelled as in the following figure.

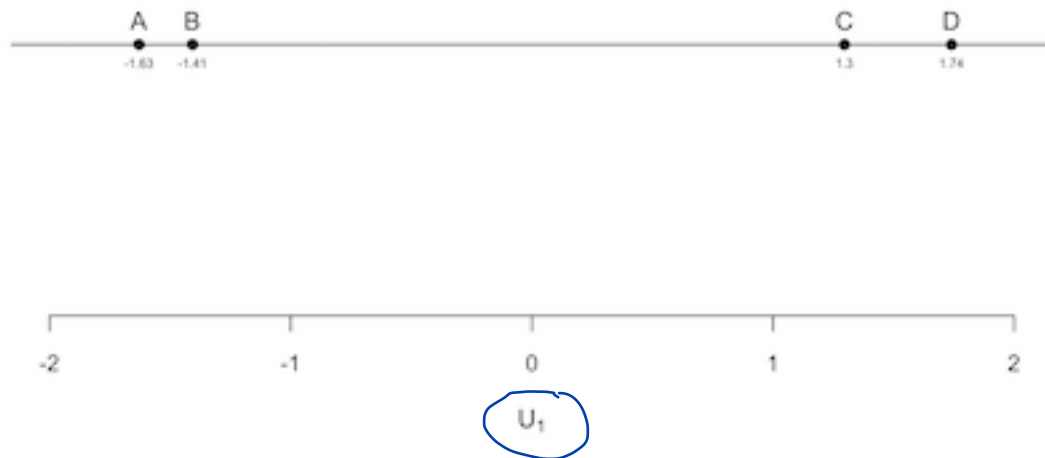


Imagine calculating all of the pairwise distances between the four points shown in the above figure.

Then ask: **What one-dimensional representation of the data preserves those distances to the greatest extent possible?**

The answer is addressed by **multidimensional scaling (MDS)**.

The result, using the classic approach, is shown in the figure below.



Exercise: What is the practical motivation behind trying to find a low-dimensional representation that preserves distances?

We believe that "distance" is a good measure of "dissimilarity." Objects which are similar are close together, objects which are dissimilar are far apart.

I want a low-dimensional rep. keeps similar objects close together and dissimilar objects far apart

Comment: There is actually a direct connection between PCA and the distance-preserving idea presented above.

When using the first k principal components to represent data of dimension greater than k , one is finding the k -dimensional representation that minimizes the strain.

The strain is a measure of the dissimilarity between the original and low-dimensional distances. Its definition is a little technical, and we will omit it here.

The procedure that results from the use of the strain as the loss metric is called classic multidimensional scaling.

A Python function to implement classic MDS can be found at

<http://www.nervouscomputer.com/hfs/cmdscale-in-python/>

If you were to run this procedure on the yield curve example of before, you would find that the k -dimensional representation returned matches (up to unimportant rotation) the representation given by PCA.

The more modern approach to this general problem is referred to as **multidimensional scaling (MDS)**. The criterion minimized by MDS is called the **Stress**. It takes a very natural form as follows:

$$\text{Stress}(x_1, x_2, \dots, x_n) = \left(\sum_{i \neq j} (d_{ij} - \Delta(x_i, x_j))^2 \right)^{1/2}$$

Handwritten annotations:
 - Arrows from $x_1, x_2, \dots, x_n$ point to "positions in low-dim space".
 - A circle around d_{ij} with an arrow pointing to "distance in high-dim. space".
 - An arrow from $\Delta(x_i, x_j)$ points to "distance in low-dim. space".

where d_{ij} is the "true" distance between observations i and j , and $\Delta(x_i, x_j)$ is the distance in the lower-dimensional mapping.

In Python, multidimensional scaling is implemented through function MDS from `scikit-learn`. (This function defines Stress without the outer square root.)

Although in “classical” MDS the distances are restricted to Euclidean, the general concept of **metric MDS** can use any distance d_{ij} which is a properly defined distance metric.

This opens up a broad range of possibilities as to how distances can be defined, and hence used for finding low-dimensional representations.

Some of the common approaches to dimension reduction are MDS used in conjunction with a well-chosen distance metric.

To demonstrate some of these ideas, consider the synthetic data set presented below.

(This demonstration is from Jake Vanderplas, and is adapted from https://scikit-learn.org/stable/auto_examples/manifold/plot_compare_methods.html.)

In this figure, consider the observations are three-dimensional, and the color of an observation is intended to give the value of some important property of each observations.

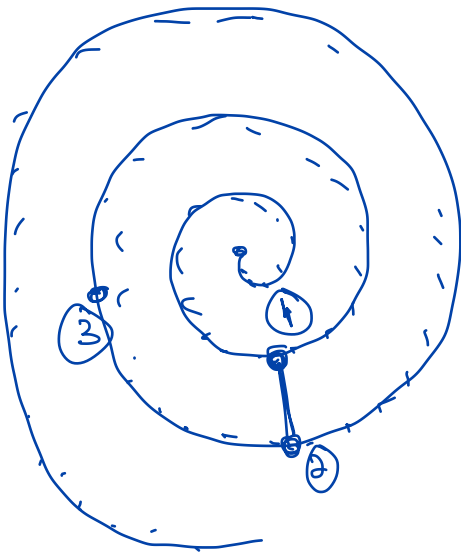
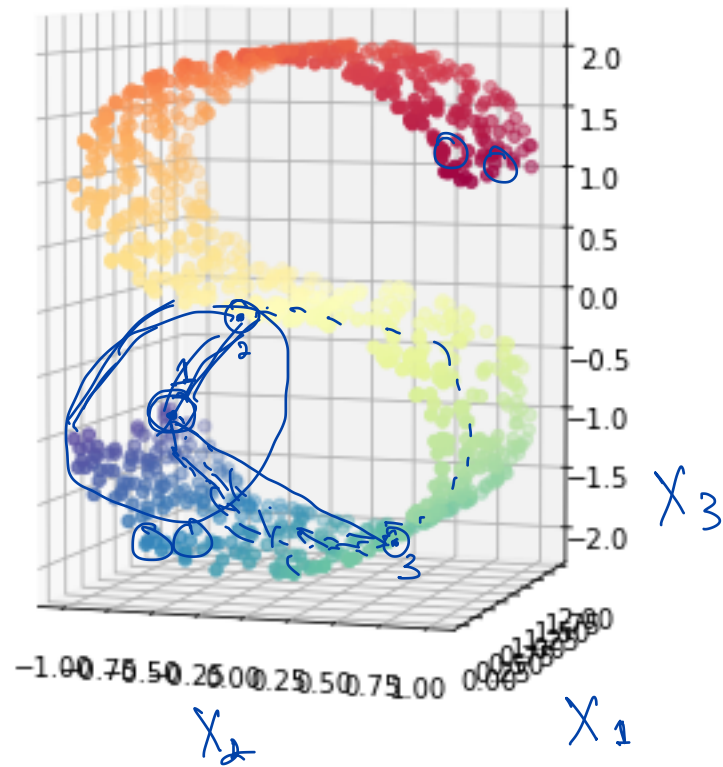
```
In [1]: import matplotlib.pyplot as plt
        from mpl_toolkits.mplot3d import Axes3D

        from sklearn import manifold, datasets

n_points = 1000
Scurve, Scolor = datasets.samples_generator.\
    make_s_curve(n_points, random_state=0)

fig = plt.figure(figsize=(5, 7))

ax = fig.add_subplot(111, projection='3d')
ax.scatter(Scurve[:, 0], Scurve[:, 1],
          Scurve[:, 2], c=Scolor,
          cmap=plt.cm.Spectral)
ax.view_init(4, -72)
plt.show()
```

Exercise: Describe the structure evident in the plot, and how it relates to the color. Do you think that Euclidean distance is a good measure of dissimilarity in this case?

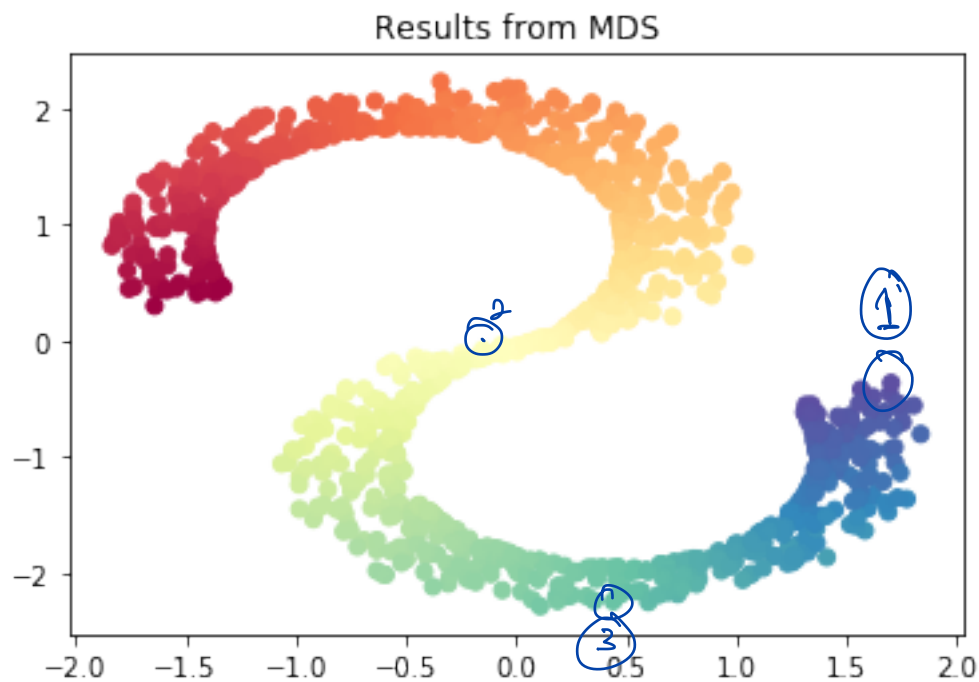
There is clear low-dimensional structure. The color can be imagined to be some important feature that evolves along the "S-curve," maybe something we want to predict.

A linear model of this response on X_1, X_2, X_3 would be a terrible choice

Euclidean distance would place ① and ② close together, even though intuitively ① and ③ should be judged more similar than ① and ②

Exercise: Comment on what happens in this example if we use metric MDS with Euclidean distance as the measure of dissimilarity.

```
In [2]: mdsout = manifold.MDS(n_components=2)
        Y = mdsout.fit_transform(Scurve)
        plt.scatter(Y[:, 0], Y[:, 1], c=Scolor,
                    cmap=plt.cm.Spectral)
        plt.title("Results from MDS")
        plt.show()
```



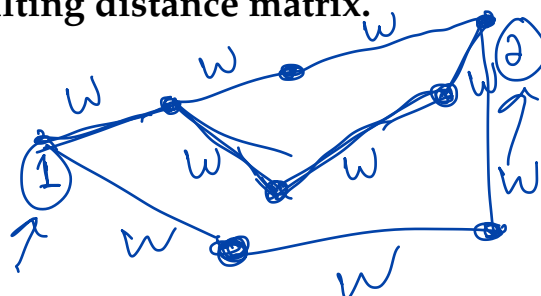
Isomap

As stated above, some approaches to dimension reduction attempt to construct an improved measure of “dissimilarity” and incorporate that into metric MDS. The above example is good illustration of why that can be necessary.

Consider the following observation: Although Euclidean distance is not a useful **global** measure of dissimilarity, it is useful on **local** scales. In other words, Euclidean distance could be used to identify two observations which are very similar.

Isomap is an approach to nonlinear dimension reduction built on MDS with a well-chosen **geodesic distance** between points. The steps:

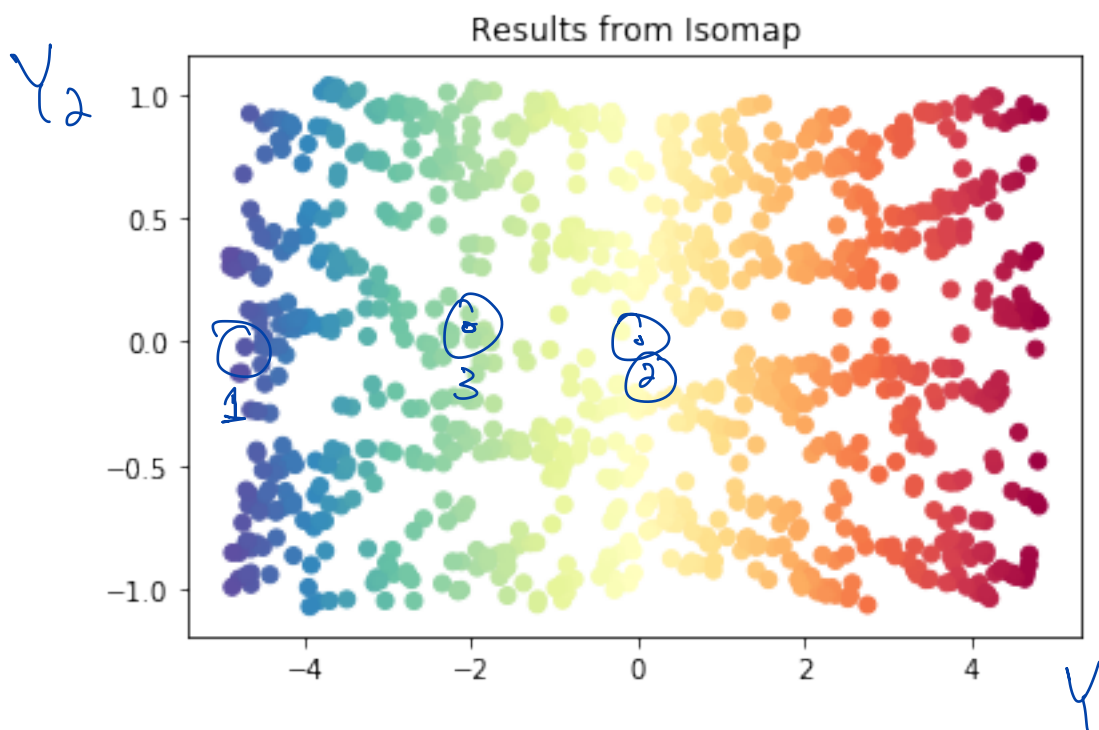
1. **Connect all “neighboring” observations.** This can be achieved by finding all observations within distance ϵ or by find the K nearest neighbors of each observation. Using Euclidean distance
2. **Assign weights to connected observations.** The weight is equal to the Euclidean distance separating them.
3. **Calculate the geodesic distance between all pairs of points.** The shortest path between points using the provided weights. d_{ij}
4. **Use metric MDS on the resulting distance matrix.**



Let's return to our synthetic example above. Note that in this case $K = 10$ is utilized. Note that choosing K too small creates the risk of having a "fragmented" graph in which there are observations with no connecting path.

In [3]: $Y =$ manifold.Isomap(n_neighbors=10, n_components=2).fit_transform(Scurve)

```
plt.scatter(Y[:, 0], Y[:, 1], c=Scolor,
            cmap=plt.cm.Spectral)
plt.title("Results from Isomap")
plt.show()
```



Exercise: Comment on the results from this dimension reduction. What would happen if only the first dimension was utilized in further analyses?

If color is the important quantity, then first dimension capture the important info.